

Dr. L. B. Ramesh

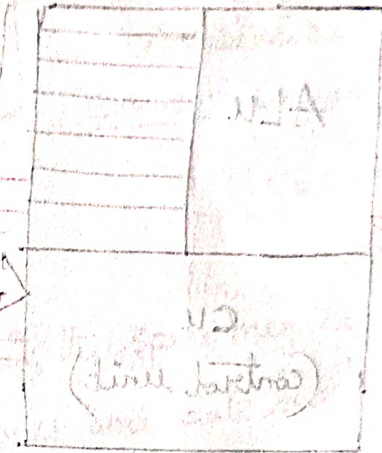
* write a program to transfer byte data from location 0x20000000 to location 0x20000010 *

Ans:

```
export __main
area prg1, code, readonly
```

```
__main
    ldr r0, =0x20000000
    ldr r1, =0x20000010
    ldrb r2, [r0]
    strb r2, [r1]
```

here b here
end



* Write a program to transfer byte from Location x to y location *

```
export __main
area prg1, code, readonly
```

```
__main
    ldr r0, =x
    ldr r1, =y
    ldrb r2, [r0]
    strb r2, [r1]
```

here

b here

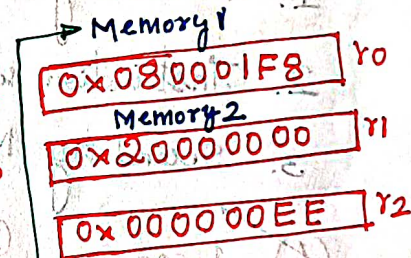
area x, data, readonly

; align 4

dcb 0xee

area y, data, readwrite

end



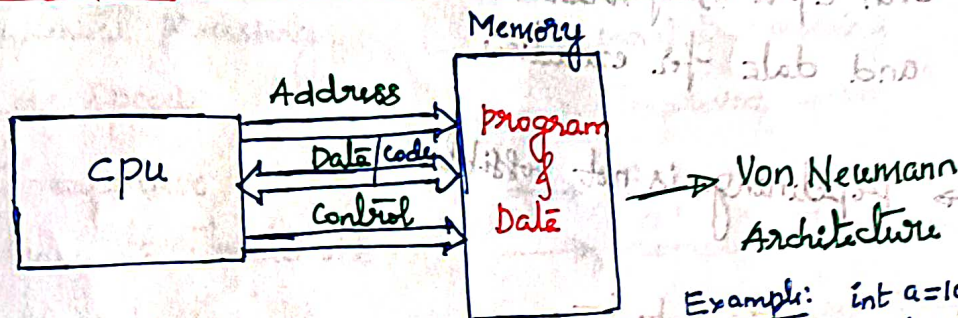
NOTE: Generally there are two states in ARM ISA.

1) ARM state: 32-bit Instructions

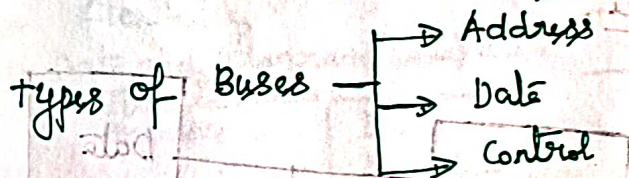
2) Thumb state: 16-bit Instructions

In lower processors like ARM7, ARM9, switching of states is needed.

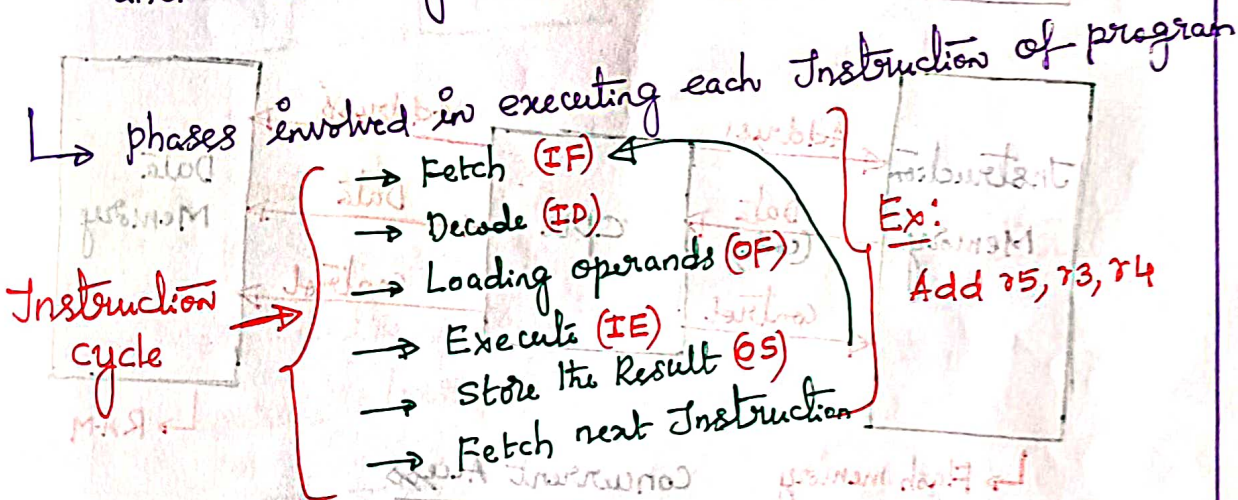
* Von Neumann Architecture and Harvard Architecture:



Example: $\text{int } a=10$
 $b=20$ } data
 $c=a+b$ } program



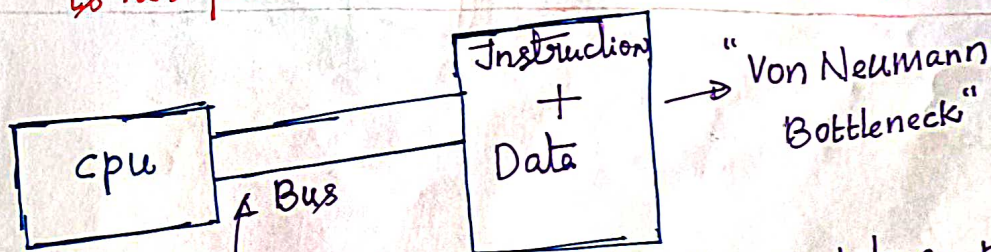
Instructions and data are stored in the same memory and accessed by the same address and data bus.



Memory Accesses can be time consuming

Both Instructions and data uses the same bus for accessing so we can't access them concurrently

It works in sequential manner, parallel processing is not possible in Von-Neumann Architecture



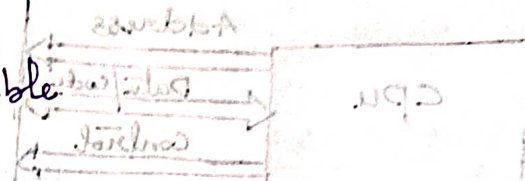
At any time either Instruction or data can pass through the bus

LBR

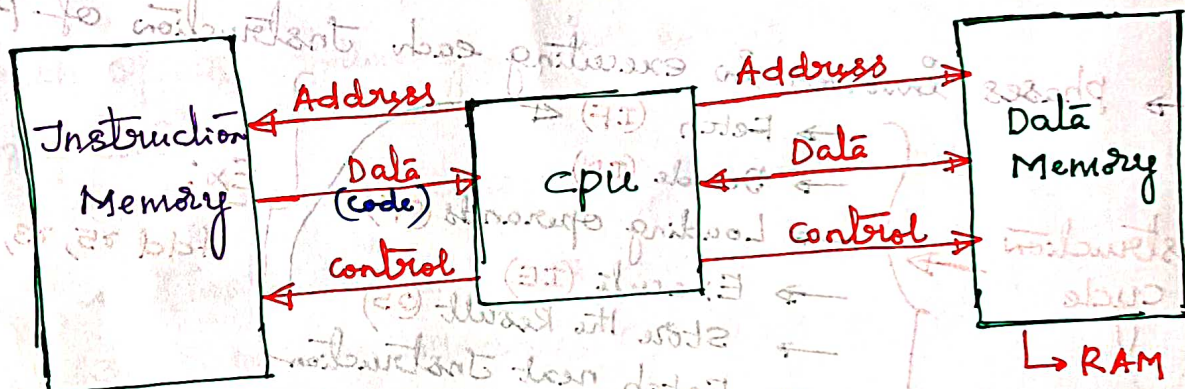
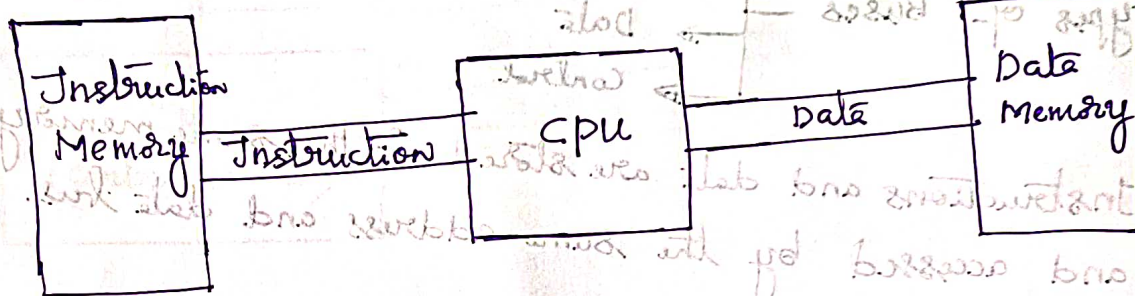
KBR

↳ Even CPU is faster but it has to wait for instructions and data for execution.

↳ pipelining is not possible.



* Harvard Architecture:



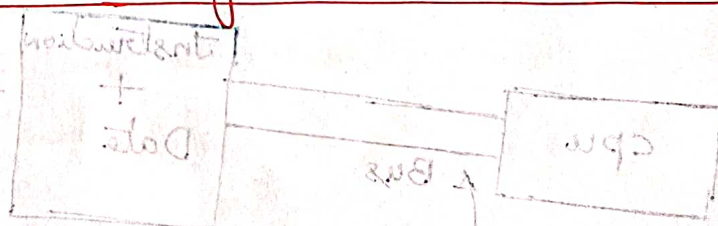
↳ Flash memory

Concurrent Access

↳ RAM

↳ Harvard Architecture results in Improved performance speed.

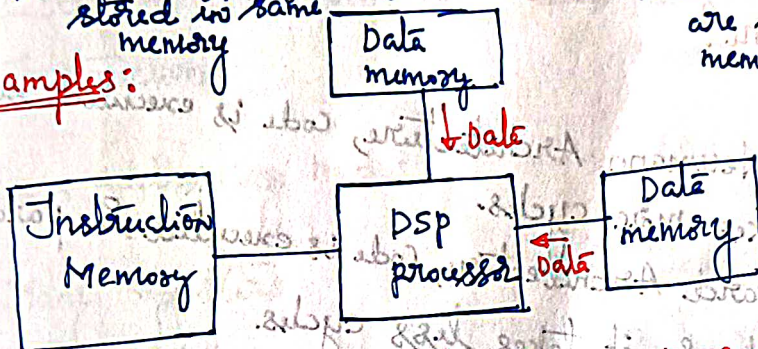
* In Von-Neumann, Data & code in one cycle and In Harvard, data and code in one cycle



Von Neumann

- sequential processing
- Less speed
- Inexpensive
- Easy to Implement
- Flexibility
- Data and program (code) are stored in same memory

Examples:

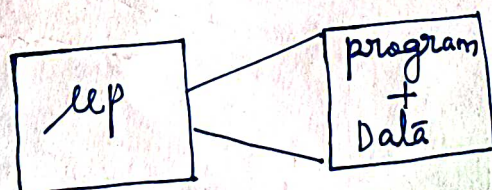
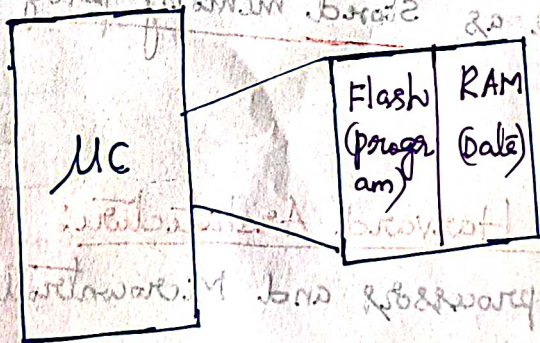


Harvard

- parallel processing
- Improved speed
- Expensive
- Complex Implementation
- Fixed memory spaces
- Data and program (code) are stored in different memory

→ DSP processors use the Harvard architecture with more than one data memory.

→ Most of the MCs use the Harvard architecture (separate memory & bus)



NOTE: Most of the modern systems use neither pure-VonNeumann nor pure Harvard architecture. Now, Modified Harvard architecture being used

RISC Vs CISC

RISC

CISC

→ Reduced Instruction set computer processor

→ Complex Instruction set computer processor

→ Fixed Instruction size

→ variable Instruction size

→ Fetch time is same for all Instructions

→ Fetch time is Vary with respect to Instructions

→ Less addressing modes

→ More addressing modes.

→ Many Registers

→ Less Registers

→ Register - to - Register

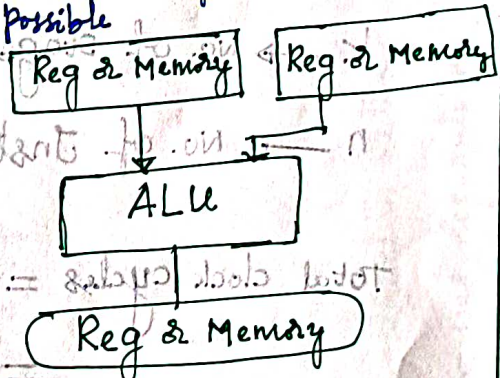
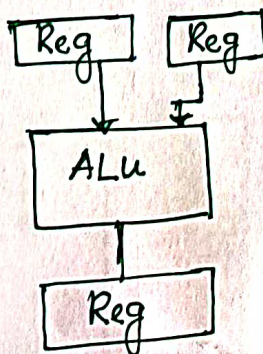
→ Register - to - memory

* Types of CPU design

→ RISC
→ CISC

Arithmetic operations only

and Memory to - register arithmetic operations are possible



→ Easy to Implement using hardwired control unit

→ Difficult to Implement using hardwired control unit

* Byte ordering

→ Little Endian

→ Big Endian

* Pipelining:

program consists of 8 Instructions, there are five stages involved to execute each Instruction They are **IF, ID, OF, IE, OS**.

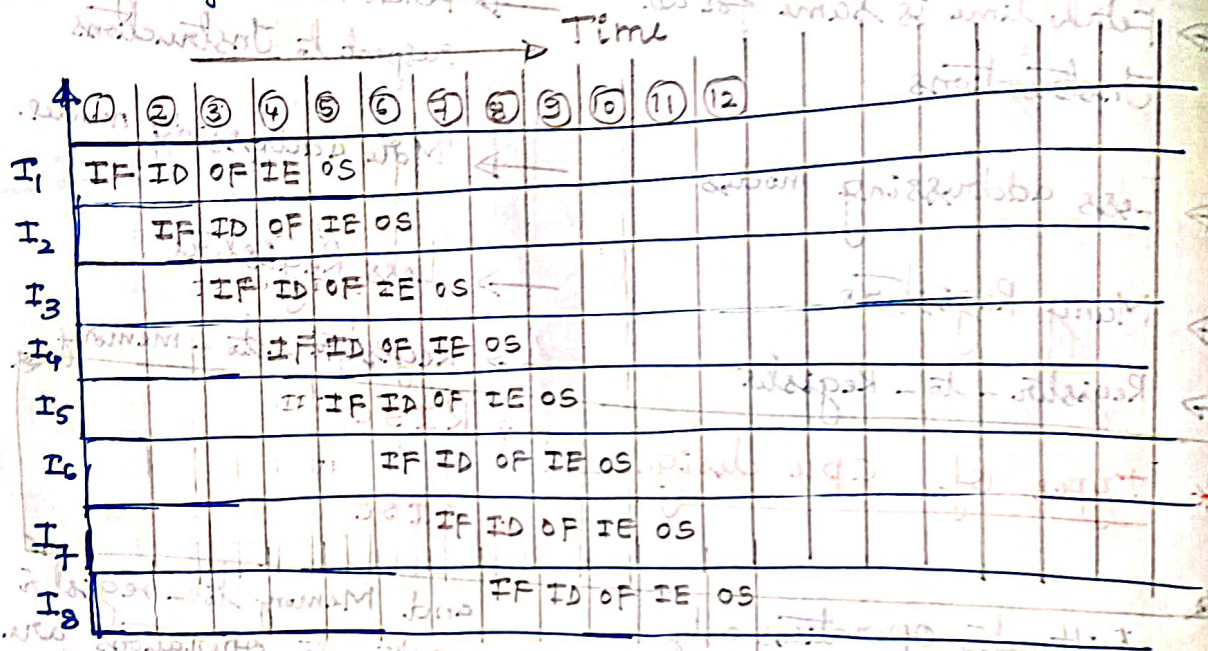
IF → Instruction fetch

ID → Instruction decode

OF → operands fetch / data fetch.

IE → Instruction Execute

OS → operands store / store the Result.



$k \rightarrow$ No. of stages

$n \rightarrow$ No. of Instructions

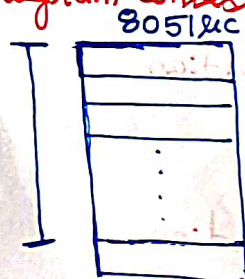
$$\text{Total clock cycles} = k + (n - 1)$$

$$= 5 + (8 - 1)$$

$$= 12 \text{ cycles}$$

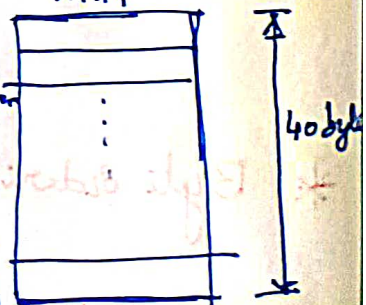
* NOTE: Comparison of 8051 μC and ARM μC

Example: program consists of 10 Instructions



10 bytes of space since each Instruction of one byte long

each Instruction takes 4 bytes long.
 $4 \times 10 = 40 \text{ bytes}$

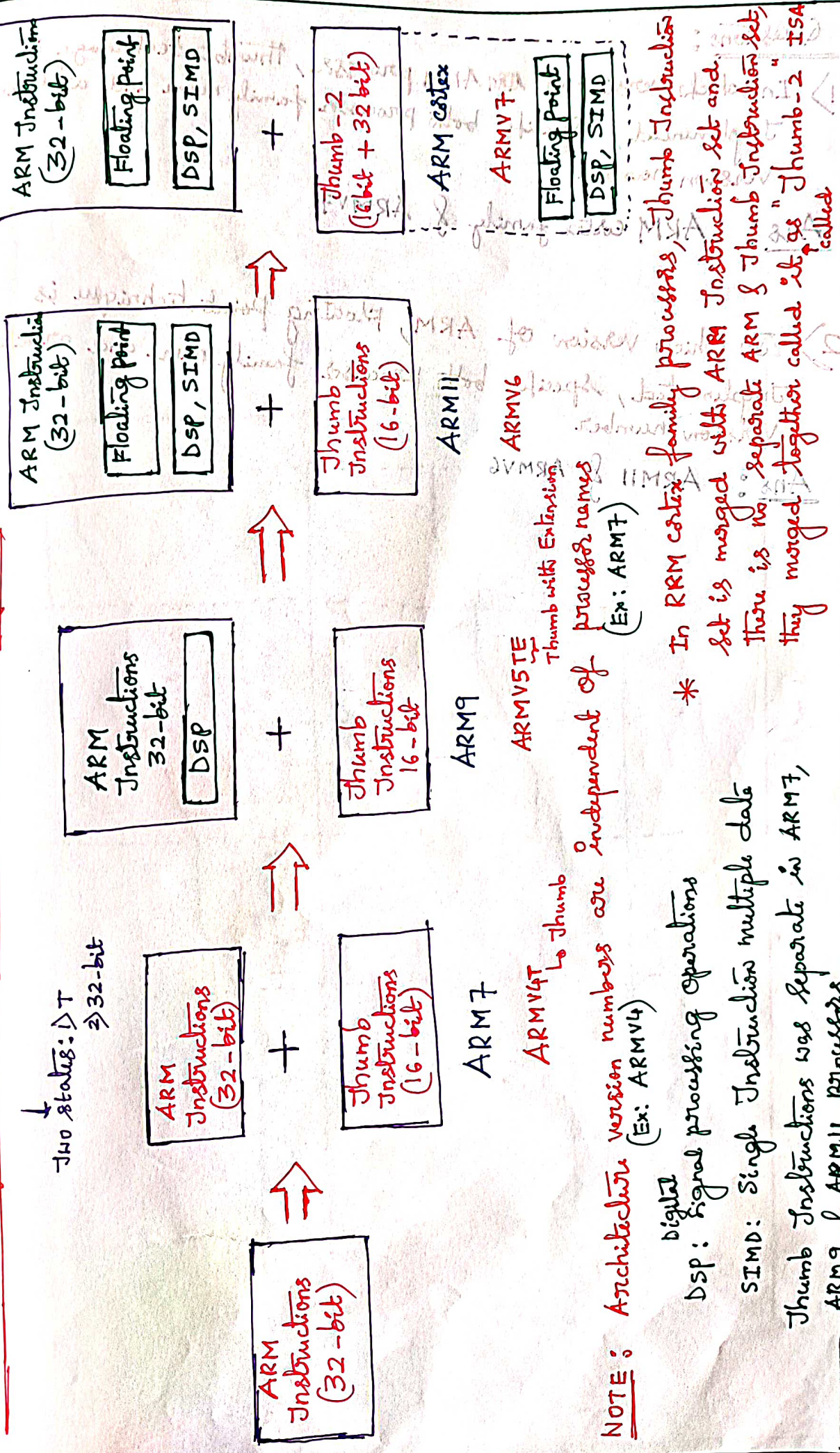


one state 32 bit + 16 bit

Dr. LBR

* Evaluation of ARM Instruction set Architecture (ISA):

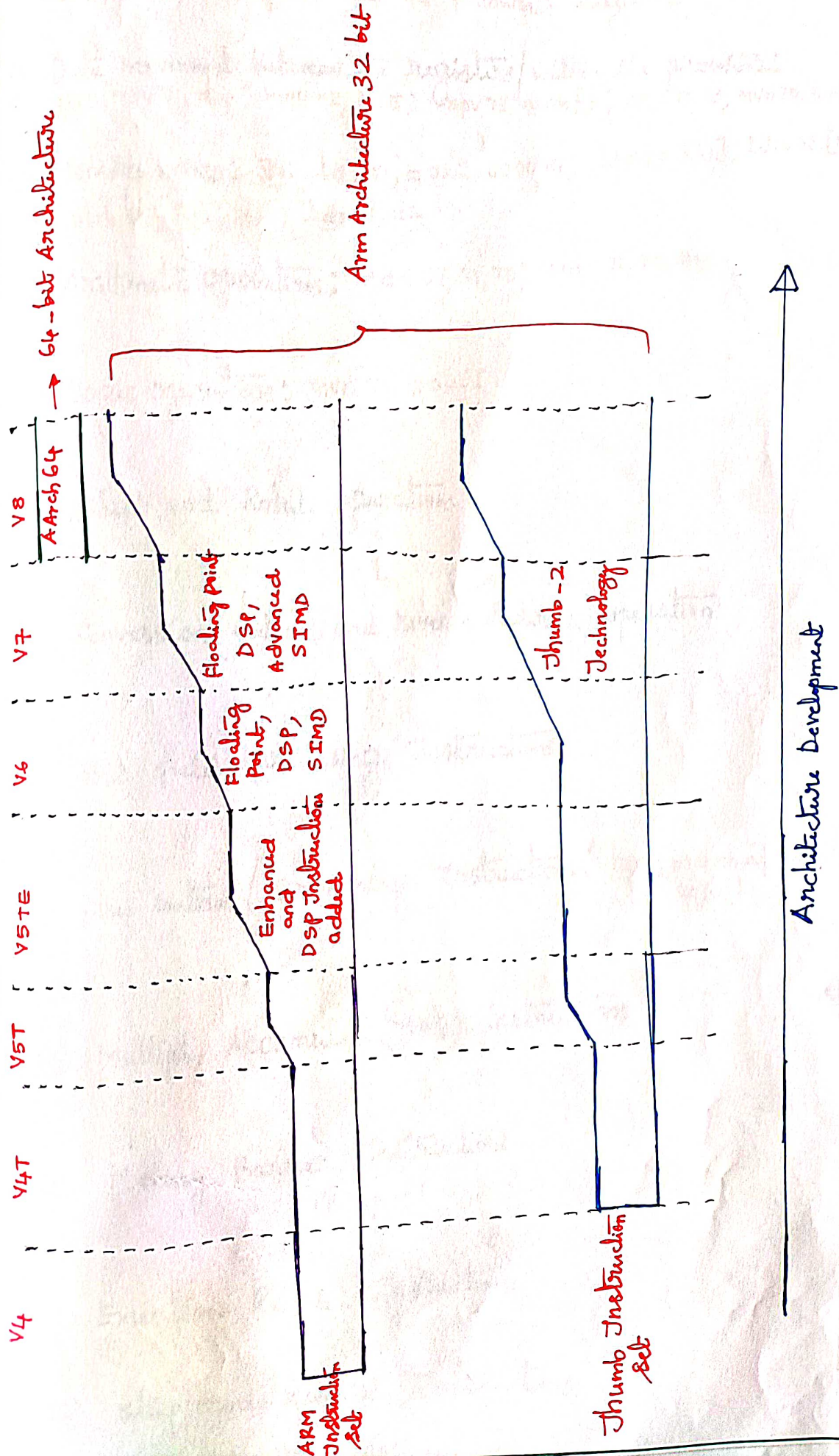
Two states: DT
 ≥ 32-bit



NOTE: Architecture version numbers are independent of processor names (Ex: ARM7)

- DSP: Digital signal processing operations
- SIMD: Single Instruction multiple data
- Thumb Instructions was separate in ARM7, ARM9 & ARM11 processors

* In RRM cortex family processors, Thumb Instructions set is merged with ARM Instructions set and there is no separate ARM & Thumb Instructions set, they merged together called it as "Thumb-2 ISA" called



* Based on the functionality/operations, Instruction set of cortex-M processor are classified into the following categories.

- 1) Data movement between the registers/within the processor:
Ex: `mov r1, #4; mov r2, #15; mov r3, #0xff; mvn r0, r3, mvn r0, #0xff`
- 2) Memory Access: Ex: `ldr r1, =0x20000000; ldr r0, [r1], ldrb r3, [r1], #1`
`strb r3, [r2], #1; ldr r1, =x`
- 3) Arithmetic operations: `Add r3, r1, r0; subs r0, r0, #1`
- 4) Logic operations: `mvn r0, #0xff`
- 5) Shift and Rotate operations
- 6) Conversion (extend and reverse ordering) operations
- 7) Bit field processing Instructions
- 8) Flow control / Branching Instructions: `bne (branch if not equal)`
- 9) Multiply Accumulate (MAC) Instructions
- 10) Memory Barrier Instructions
- 11) Exception Related Instructions
- 12) sleep mode related Instructions
- 13) Enhanced Dsp Instructions

* Architecture:

↳ conceptual design of fundamental operational structure

↳ CPU design

↳ Instructions

↳ Addressing modes

↳ Data format

* CPU Arithmetic

- Decimal Arithmetic
- Binary Arithmetic
 - Fixed-point
 - Floating-point

*

Numbers

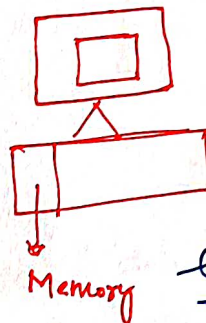
Text

Audio

Images

Video

Storage
Data



Data

- Data can be represented in many forms
- Irrespective of data type
- Stored in the memory
- Retrieved back into original form

↳ The description of binary numbers in the exponential form is called floating-point representation.

↳ Representation of floating point number is not unique. Ex:
The number 55.66 can be represented as 5.566×10^1 , 0.5566×10^2 , 0.05566×10^3 , and so on.

Dr. KBR

* Write a program to Interchange (exchange) between two blocks of memory.

Ans:

— main

area prg1, code, readonly

ldr r0, =0x20000000

ldr r1, =0x20000010

mov r2, #4

loop1 ldr r3, [r0]

ldr r4, [r1]

str r3, [r1], #4

str r4, [r0], #4

subs r2, r2, 1

cmp r2, #0

bne loop1

here

b here

end

* Write a program to transfer block of data from ROM locations to RAM locations.

Ans:

export — main

area prg1, code, readonly

— main ldr r0, =x

ldr r1, =y

mov r2, #3

loop1

ldr r3, [r0], #4

str r3, [r1], #4

subs r2, r2, #1

cmp r2, #0

bne loop1

here

b here

; dcb 0xaa, 0xbb, 0xcc

area x, data, readonly

dcd 0xaaaaaaaa, 0xbbbbbbbb, 0xcccccccc

area y, data, readwrite

Dr. LBR

* Multiplication Instruction:

umull (unsigned multiply long): $[RdH, RdL] = Rm \times Rs$

Ex: 32 bit $\times$ 32 bit = 64 bit long

$r0 = 0 \times 00000000$

$r1 = 0 \times 00000000$

$r2 = 0 \times f0000002$

$r3 = 0 \times 00000002$

$f = 15$

$2 \times 15 = 30$ delay

16 | 30
1 - 16
1E

umull r0, r1, r2, r3 : $[r1, r0] = r2 \times r3$

Result

$r2 \times r3 =$

r1	r0
----	----

Lower product

Higher product

0x00000001	0xe0000004
------------	------------

r1

r0

Low order product

High order product

NOTE: mul $\rightarrow$ Multiply

1) mul rd, rm, rs

2) muls rd, rm, rs

MLA $\rightarrow$ Multiply and Accumulate

Mul $\rightarrow$ Multiply

Dr. LBR

* Write ALP to Evaluate the Expression:

$$Z = \sum_{i=0}^{n-1} x_i y_i$$

$$= x_0 y_0 + x_1 y_1 + x_2 y_2 + x_3 y_3 + \dots + x_{n-1} y_{n-1}$$

Export main

area prg2, code, readonly

mov r0, #5 ; n=5

ldr r1, =x

ldr r2, =y

mov r5, #0 ; lower product

mov r6, #0 ; Higher product

Initial sum=0

loop1

ldr r3, [r1], #4 ; content of r1 is incremented by 4

ldr r4, [r2], #4 ; content of r2 is incremented by 4

umlal r5, r6, r3, r4 ; $[r6, r5] = [r6, r5] + [r3 \times r4]$

subs r0, #1

bne loop1

here

b here

area x, data, readonly

dcd 1, 2, 3, 4, 5

area y, data, readonly

dcd 1, 2, 3, 4, 5

end

r6	r5
0x00000000	0x00000037

* Data Transfer / movement Instructions:

1) `mov r0, r1 ; [r1] → [r0]`

2) `mov r0, #0x25` ; Immediate data mov Instruction

NOTE: only 8-bits data can be moved immediately

3) `movs r0, r1` ; Move Instruction which affect the flag

Ex: if $r_1 = 0$, after move $r_0 = 0$, but it will affect the zero flag, $Z = 1$;

if r_1 is negative, after move, $r_0 = r_1$, but it will affect the negative flag, $N = 1$;

4) Conditional MOV Instructions: It performs move operation based on condition

Ex: `MOVCC r0, r1` ;

It moves the data from r_1 to r_0 only if carry flag is zero.

Example:

CS - Carry set, $C = 1$

CC - Carry clear, $C = 0$

VS - overflow set, $V = 1$

VC - overflow clear, $V = 0$

EQ - Equal, $Z = 1$

NE - Not Equal, $Z = 0$

`MOVCS r0, r1` ;

It performs move based on previous flag status and it will update the status flags after execution.

5) MOV NOT Instruction:

`MVN r0, r1`

`MVNVSS r0, r1`

Example:

ldr r0, =0xffffffff ;
 adds r0, #1

r0 0xffffffff

r0 0x00000000

N=0; Z=1; C=1; V=0

Example:

ldr r0, =0xffffffff ;
~~comp~~ cmn r0, #1

r0 0xffffffff

r0 0xffffffff

N=0; Z=1; C=1; V=0

cmn: compare negated
 flags set as a result
 of r0 + 1.

Here, flags will change
 according to the result
 [r0] + 1. Content of
 register r0 will remain
 same.

Example:

mov r0, #0x47
 mov r1, #0x25
 subs r0, r1

r0 0x00000047

r1 0x00000025

After Execution of Subs

r0 0x00000022

r1 0x00000025

N=0; Z=0; C=1; V=0

Example:

mov r0, #0x47
 mov r1, #0x25
 cmp r0, r1

After Execution of cmp r0, r1

r0 0x00000047

r1 0x00000025

N=0; Z=0; C=1; V=0

* Features of cortex-M3 and cortex-M4 processors have:

↳ Three-stage pipeline design ↳ processor is a 32-bit RISC processor

↳ Harvard bus architecture with unified memory space. Instructions and data use the same address space.

↳ Thumb-2 Technology ISA (Instruction set Architecture)
(Mixture of 16-bit and 32-bit Instructions)

↳ 32-bit addressing: supporting of 4 GB of memory space
NOTE: $2^{32} \approx 4 \text{ GB}$

↳ on chip bus Interfacing based on ARM AMBA (Advanced microcontroller bus Architecture) technique which allows higher throughput.

↳ Having NVIC (Nested Vectored Interrupt Controller) system
→ It supports up to 240 Vectored Interrupts

↳ Having MPU (Memory protection unit) system

↳ It supports bit band memory mapping. i.e. bit data access.
↳ Bit banding

NOTE: Three stage pipeline consists of three stages: Fetch, Execute and decode

program consists of five Instructions, comparison of performance of Non-pipelined architecture with ARM pipelined architecture with three stages of Execution.

	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	⑪	⑫	⑬	⑭	⑮	
Instr 1	F1	D1	E1													Normal Execution without Pipelining takes 15 clock cycles
2				F2	D2	E2										
3							F3	D3	E3							
4										F4	D4	E4				
5													F5	D5	E5	
Instr 1	F1	D1	E1													ARM with Pipelining takes 7 clock cycles
Instr 2		F2	D2	E2												
Instr 3			F3	D3	E3											
Instr 4				F4	D4	E4										
Instr 5					F5	D5	E5									

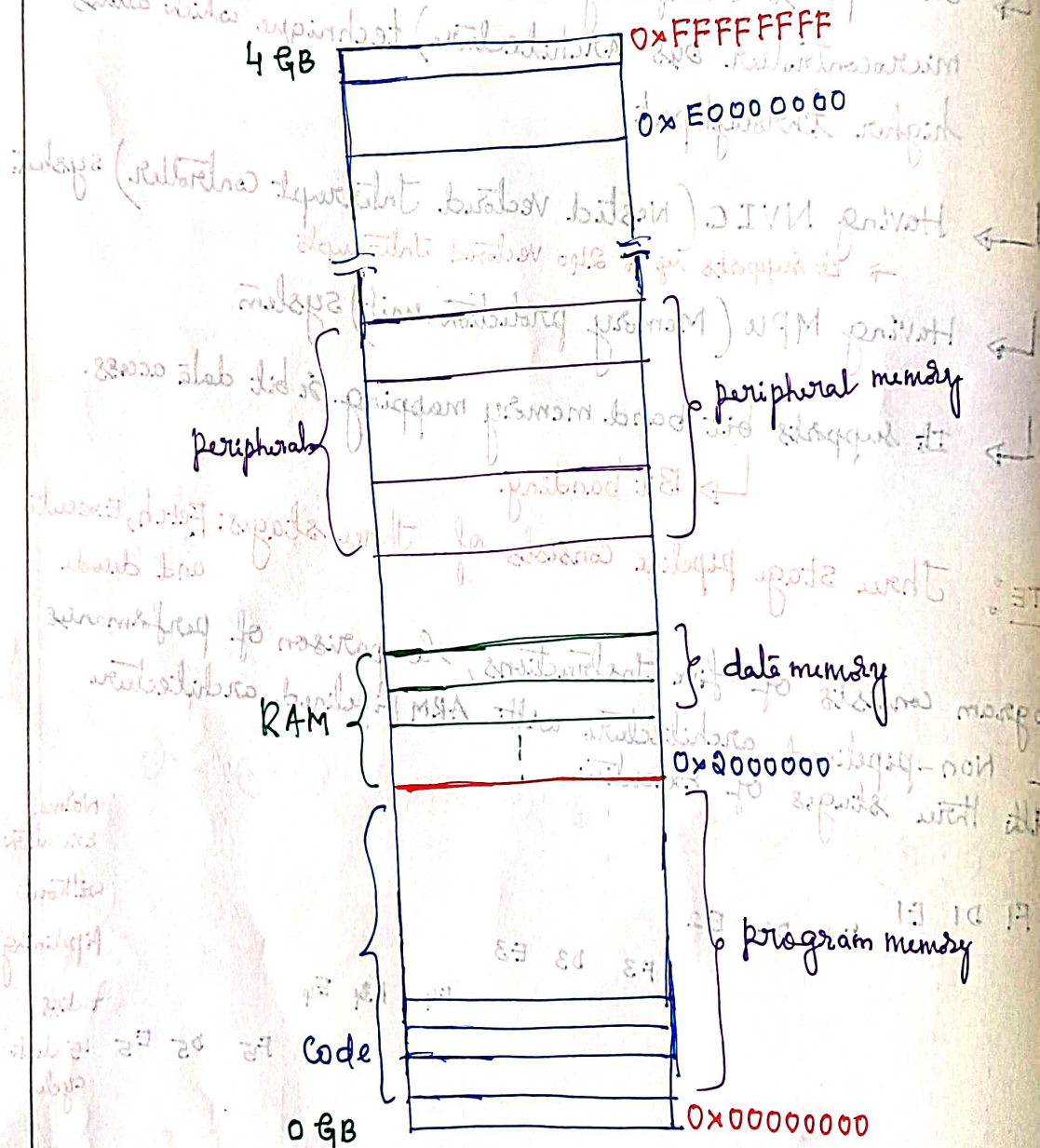
∴ No of machine cycles = $N + 2$
N → No of Instructions.

∴ It is faster than Non-pipelined architecture.

* Harvard Architecture with unified memory space:

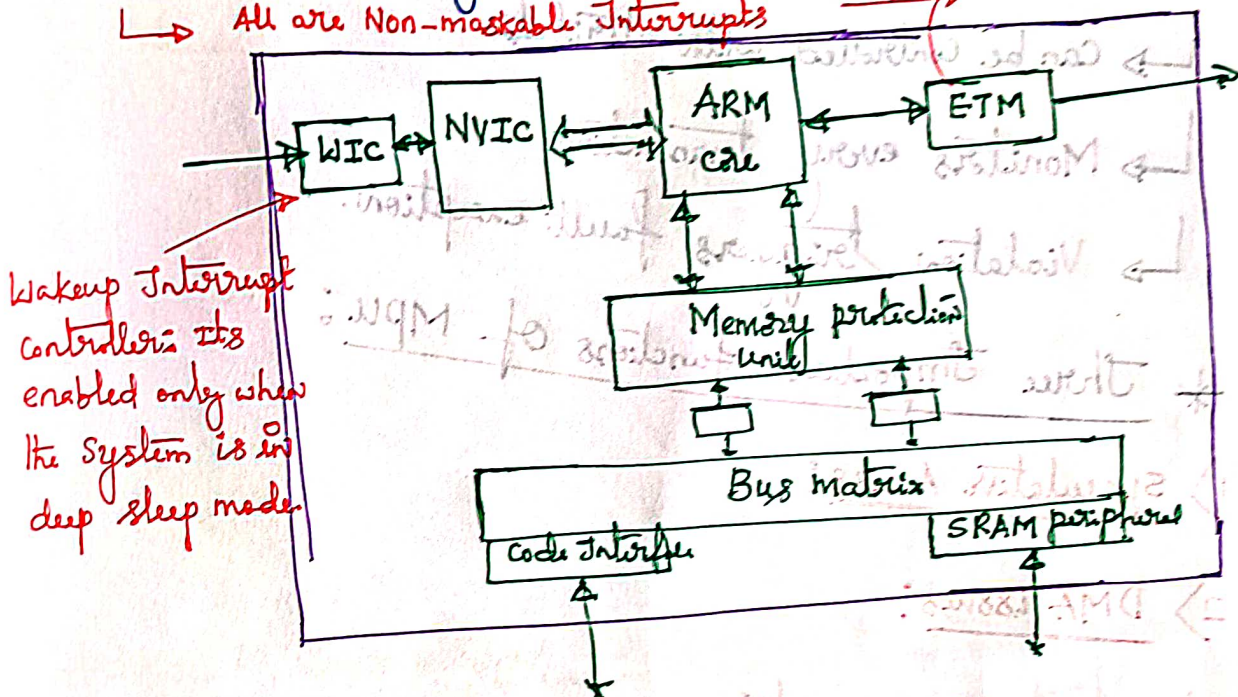
↳ Memory mapped which means that given an address and appear to be part of the same memory as the instructions and data.

↳ The program memory and data memory appear to be in the same memory despite being physically separate. This is known as unified memory map.



* Nested Vectored Interrupt Controller (NVIC):

- ↳ It supports up to 240 Vectored Interrupts
- ↳ It supports multiple levels of Interrupt priorities (from 8 to 256 levels). This feature makes the cortex-M processor suitable for real time control applications.
- ↳ We can change the priority of an interrupt dynamically.
- ↳ All are Non-maskable Interrupts



- ↳ As shown in the above block diagram, the NVIC and processor core are closely coupled to enable low latency processing and efficient/fast processing of late arriving interrupts.

NOTE: Low latency → less delay.
Low latency value is good.

KBR

* Memory protection unit [MPU] :

- ↳ It's a piece of hardware
- ↳ prevents a process from accessing unallocated region
- ↳ Allows privileged access to defined regions

↳ Can be controlled with attributes

↳ Monitors every transaction

↳ Violation triggers fault exception.

* Three Important functions of MPU :

1) Speculative Access :

2) DMA issues :

3) Task Management :

NOTE : * The MPU can be used to protect memory regions by means of defining access permissions in privileged and unprivileged access states.

* MPU is programmable, Cortex-M3/M4 processor supports eight programmable regions.

* Defines the memory Access permission

* Mainly for preventing accidental erasing of data or over writing of Instructions

* Bit Banding:

- ↳ special feature of cortex-M processors where any bit stored in SRAM or peripheral memory has a corresponding alias address which can be used to reference that bit specifically.
- ↳ In bit band region, we can manipulate a single bit of that stored value without having to deal with the whole stored value.

* Advanced Microcontroller Bus Architecture (AMBA):

- ↳ open standard on chip interconnect specification
- ↳ Defines the connection and management of functional blocks in system on chips (soc) designs.
- ↳ AMBA protocols define how functional blocks communicate with each other.